

Escuela Politécnica Nacional

Nombre: Kevin Eduardo Garcia Rodriguez

Tema: [Tarea 10] Ejercicios Unidad 04-C | Descomposición LU

Repositorio GIT: <https://github.com/Nattyrd/Metodos-Numericos-2025B>

Ejercicio 1

Realice las siguientes multiplicaciones matriz-matriz

$$\begin{array}{ll} \text{a.} \quad \begin{bmatrix} 2 & -3 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 & 5 \\ 2 & 0 \end{bmatrix} & \text{b.} \quad \begin{bmatrix} 2 & -3 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 & 5 & -4 \\ -3 & 2 & 0 \end{bmatrix} \\ \text{c.} \quad \begin{bmatrix} 2 & -3 & 1 \\ 4 & 3 & 0 \\ 5 & 2 & -4 \end{bmatrix} \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & -1 \\ 2 & 3 & -2 \end{bmatrix} & \text{d.} \quad \begin{bmatrix} 2 & 1 & 2 \\ -2 & 3 & 0 \\ 2 & -1 & 3 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -4 & 1 \\ 0 & 2 \end{bmatrix} \end{array}$$

```
In [1]: def multiplicar_matrices(A, B):
# Verifica si las matrices son compatibles
if len(A[0]) != len(B):
    raise ValueError("Las dimensiones no permiten la multiplicación")

# Inicializa la matriz resultado con ceros
resultado = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]

# Multiplicación clásica
for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
            resultado[i][j] += A[i][k] * B[k][j]

return resultado
```

```
In [2]: # Inciso a
A_a = [ [2, -3], [3, -1] ]
B_a = [ [1, 5], [2, 0] ]

resultado_a = multiplicar_matrices(A_a, B_a)
print("Resultado inciso a:")
for fila in resultado_a:
    print(fila)
```

Resultado inciso a:

[-4, 10]

[1, 15]

```
In [3]: # Inciso b
A_b = [ [2, -3], [3, -1] ]
B_b = [ [1, 5, -4], [-3, 2, 0] ]
resultado_b = multiplicar_matrices(A_b, B_b)
print("\nResultado inciso b:")
```

```
for fila in resultado_b:
    print(fila)
```

Resultado inciso b:
 [11, 4, -8]
 [6, 13, -12]

```
In [4]: # Inciso c
A_c = [ [2, -3, 1], [4, 3, 0], [5, 2, -4] ]
B_c = [ [0, 1, -2], [1, 0, -1], [2, 3, -2] ]
resultado_c = multiplicar_matrices(A_c, B_c)
print("\nResultado inciso c:")
for fila in resultado_c:
    print(fila)
```

Resultado inciso c:
 [-1, 5, -3]
 [3, 4, -11]
 [-6, -7, -4]

```
In [5]: # Inciso d
A_d = [ [2, 1, 2], [-2, 3, 0], [2, -1, 3] ]
B_d = [ [1, -2], [-4, 1], [0, 2] ]
resultado_d = multiplicar_matrices(A_d, B_d)
print("\nResultado inciso d:")
for fila in resultado_d:
    print(fila)
```

Resultado inciso d:
 [-2, 1]
 [-14, 7]
 [6, 1]

Ejercicio 2

Determine cuáles de las siguientes matrices son no singulares y calcule la inversa de esas matrices

$$\text{a. } \begin{bmatrix} 4 & 2 & 6 \\ 3 & 0 & 7 \\ -2 & -1 & -3 \end{bmatrix}$$

$$\text{c. } \begin{bmatrix} 1 & 1 & -1 & 1 \\ 1 & 2 & -4 & -2 \\ 2 & 1 & 1 & 5 \\ -1 & 0 & -2 & -4 \end{bmatrix}$$

$$\text{b. } \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & -1 \\ 3 & 1 & 1 \end{bmatrix}$$

$$\text{d. } \begin{bmatrix} 4 & 0 & 0 & 0 \\ 6 & 7 & 0 & 0 \\ 9 & 11 & 1 & 0 \\ 5 & 4 & 1 & 1 \end{bmatrix}$$

```
In [6]: import numpy as np

# Definición de matrices
A_a = np.array([
    [4, 2, 6],
    [3, 0, 7],
    [-2, -1, -3]
])

A_b = np.array([
```

```

    [1, 2, 0],
    [2, 1, -1],
    [3, 1, 1]
])

A_c = np.array([
    [1, 1, -1, 1],
    [1, 2, -4, -2],
    [2, 1, 1, 5],
    [-1, 0, -2, -4]
])

A_d = np.array([
    [4, 0, 0, 0],
    [6, 7, 0, 0],
    [9, 11, 0, 0],
    [5, 4, 1, 1]
])

# Función para verificar si una matriz es no singular e imprimir su inversa
def verificar_inversa(matriz, nombre):
    try:
        det = np.linalg.det(matriz)
        print(f"Determinante de {nombre}: {det:.2f}")
        if det != 0:
            inversa = np.linalg.inv(matriz)
            print(f"Inversa de {nombre}: \n{inversa}\n")
        else:
            print(f"{nombre} es singular. No tiene inversa.\n")
    except Exception as e:
        print(f"Error con {nombre}: {e}\n")

# Verificar todas
verificar_inversa(A_a, "A_a")
verificar_inversa(A_b, "A_b")
verificar_inversa(A_c, "A_c")
verificar_inversa(A_d, "A_d")

```

Determinante de A_a: 0.00
A_a es singular. No tiene inversa.

Determinante de A_b: -8.00
Inversa de A_b:
[[-0.25 0.25 0.25]
 [0.625 -0.125 -0.125]
 [0.125 -0.625 0.375]]

Determinante de A_c: 0.00
A_c es singular. No tiene inversa.

Determinante de A_d: 0.00
A_d es singular. No tiene inversa.

In [28]: `import numpy as np`

```

def calcular_inversa(matriz):
    """
    Calcula la inversa de una matriz cuadrada si es no singular.

```

```

Args:
    matriz: Array de numpy representando la matriz cuadrada.

Returns:
    inversa: Array de numpy con la matriz inversa.
    mensaje: String indicando si la matriz es singular.
"""
try:
    # Verificar si la matriz es cuadrada
    if matriz.shape[0] != matriz.shape[1]:
        return None, "La matriz no es cuadrada"

    # Calcular el determinante
    det = np.linalg.det(matriz)

    if np.isclose(det, 0):
        return None, "La matriz es singular (determinante = 0)"

    # Calcular la inversa
    inversa = np.linalg.inv(matriz)
    return inversa, "Matriz inversa calculada correctamente"

except Exception as e:
    return None, f"Error: {str(e)}"

```

```

In [29]: # Definición de Las matrices
B = np.array([
    [1, 2, 0],
    [2, 1, -1],
    [3, 1, 1]
])

D = np.array([
    [4, 0, 0, 0],
    [6, 7, 0, 0],
    [9, 11, 1, 0],
    [5, 4, 1, 1]
])

# Calcular inversas
inversa_B, mensaje_B = calcular_inversa(B)
inversa_D, mensaje_D = calcular_inversa(D)

# Resultados
print("=== Matriz B ===")
print(mensaje_B)
if inversa_B is not None:
    print("Inversa de B:")
    print(np.round(inversa_B, 6)) # Redondeamos a 6 decimales para mejor visual

print("\n=== Matriz D ===")
print(mensaje_D)
if inversa_D is not None:
    print("Inversa de D:")
    print(np.round(inversa_D, 6))

```

```

=== Matriz B ===
Matriz inversa calculada correctamente
Inversa de B:
[[-0.25  0.25  0.25 ]
 [ 0.625 -0.125 -0.125]
 [ 0.125 -0.625  0.375]]

=== Matriz D ===
Matriz inversa calculada correctamente
Inversa de D:
[[ 0.25      0.      0.      0.      ]
 [-0.214286  0.142857 -0.      -0.      ]
 [ 0.107143 -1.571429  1.      -0.      ]
 [-0.5       1.      -1.      1.      ]]

```

Ejercicio 3

Resuelva los sistemas lineales 4 x 4 que tienen la misma matriz de coeficientes

Sistema de ecuaciones 1

- $x_1 - x_2 + 2x_3 - x_4 = 6$
- $x_1 - x_3 + x_4 = 4$
- $2x_1 + x_2 + 3x_3 - 4x_4 = -2$
- $-x_2 + x_3 - x_4 = 5$

Sistema de ecuaciones 2

- $x_1 - x_2 + 2x_3 - x_4 = 1$
- $x_1 - x_3 + x_4 = 1$
- $2x_1 + x_2 + 3x_3 - 4x_4 = 2$
- $-x_2 + x_3 - x_4 = -1$

```

In [1]: import numpy as np

# Matriz de coeficientes (común a ambos sistemas)
A = np.array([
    [1, -1, 2, -1],
    [1, 0, -1, 1],
    [2, 1, 3, -4],
    [0, -1, 1, -1]
])

# Términos independientes del primer sistema
b1 = np.array([6, 4, -2, 5])

# Términos independientes del segundo sistema
b2 = np.array([1, 1, 2, -1])

# Resolver ambos sistemas

```

```

try:
    # Solución del primer sistema
    x1 = np.linalg.solve(A, b1)
    print("Solución del primer sistema (6,4,-2,5):")
    print(f"x1 = {x1[0]:.2f}, x2 = {x1[1]:.2f}, x3 = {x1[2]:.2f}, x4 = {x1[3]:.2f}")

    # Solución del segundo sistema
    x2 = np.linalg.solve(A, b2)
    print("\nSolución del segundo sistema (1,1,2,-1):")
    print(f"x1 = {x2[0]:.2f}, x2 = {x2[1]:.2f}, x3 = {x2[2]:.2f}, x4 = {x2[3]:.2f}")

except np.linalg.LinAlgError:
    print("La matriz es singular y no puede resolverse de forma única.")

```

Solución del primer sistema (6,4,-2,5):
x1 = 3.00, x2 = -6.00, x3 = -2.00, x4 = -1.00

Solución del segundo sistema (1,1,2,-1):
x1 = 1.00, x2 = 1.00, x3 = 1.00, x4 = 1.00

Ejercicio 4

Encuentre los valores de A que hacen que la siguiente matriz sea singular

```

In [2]: import numpy as np
import sympy as sp

# Definimos alpha como símbolo
α = sp.symbols('α')

# Creamos la matriz simbólica
A = sp.Matrix([
    [1, -1, α],
    [2, 2, 1],
    [0, α, -3/2]
])

# Calculamos el determinante
det_A = A.det()

# Resolvemos det(A) = 0
soluciones = sp.solve(det_A, α)

print("El determinante de la matriz es:", det_A)
print("\nLos valores de α que hacen la matriz singular son:", soluciones)

```

El determinante de la matriz es: $2\alpha^2 - \alpha - 6.0$

Los valores de α que hacen la matriz singular son: $[-1.50000000000000, 2.00000000000000]$

```

In [11]: x = -1.5

A1 = np.array([
    [1, -1, x],
    [2, 2, 1],
    [0, x, -1.5]
])
print("\nDeterminante cuando α = -1:", np.linalg.det(A1))

```

```

x = 2
A2 = np.array([
    [1, -1, x],
    [2, 2, 1],
    [0, x, -1.5]
])
print("Determinante cuando α = 4/3:", np.linalg.det(A2))

```

Determinante cuando α = -1: 0.0

Determinante cuando α = 4/3: 0.0

Ejercicio 5

Resuelva los siguientes sistemas lineales

$$\mathbf{a.} \quad \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 & -1 \\ 0 & -2 & 1 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix}$$

```

In [12]: import numpy as np

# Sistema (a)
L_a = np.array([
    [1, 0, 0],
    [2, 1, 0],
    [-1, 0, 1]
])

U_a = np.array([
    [2, 3, -1],
    [0, -2, 1],
    [0, 0, 3]
])

b_a = np.array([2, -1, 1])

# Resolver L·y = b
y_a = np.linalg.solve(L_a, b_a)

# Resolver U·x = y
x_a = np.linalg.solve(U_a, y_a)

print("Solución del sistema (a):")
print(f"x1 = {x_a[0]:.2f}, x2 = {x_a[1]:.2f}, x3 = {x_a[2]:.2f}")

```

Solución del sistema (a):

x1 = -3.00, x2 = 3.00, x3 = 1.00

$$\mathbf{b.} \quad \begin{bmatrix} 2 & 0 & 0 \\ -1 & 1 & 0 \\ 3 & 2 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \\ 0 \end{bmatrix}$$

```

In [13]: # Sistema (b)
L_b = np.array([
    [2, 0, 0],
    [-1, 1, 0],
    [3, 2, -1]
])

```

```

U_b = np.array([
    [1, 1, 1],
    [0, 1, 2],
    [0, 0, 1]
])

b_b = np.array([-1, 3, 0])

# Resolver  $L \cdot y = b$ 
y_b = np.linalg.solve(L_b, b_b)

# Resolver  $U \cdot x = y$ 
x_b = np.linalg.solve(U_b, y_b)

print("\nSolución del sistema (b):")
print(f"x1 = {x_b[0]:.2f}, x2 = {x_b[1]:.2f}, x3 = {x_b[2]:.2f}")

```

Solución del sistema (b):

x1 = 0.50, x2 = -4.50, x3 = 3.50

```

In [14]: # Verificación para (a)
A_a = L_a @ U_a
x_a_verif = np.linalg.solve(A_a, b_a)
print("\nVerificación sistema (a):", np.allclose(x_a, x_a_verif))

# Verificación para (b)
A_b = L_b @ U_b
x_b_verif = np.linalg.solve(A_b, b_b)
print("Verificación sistema (b):", np.allclose(x_b, x_b_verif))

```

Verificación sistema (a): True

Verificación sistema (b): True

Ejercicio 6

Factorice las siguientes matrices en la descomposición LU mediante el algoritmo de factorización LU con $l_{ii} = 1$ para todas las i .

```

In [15]: import numpy as np

def lu_decomposition(A):
    """
    Realiza la descomposición LU de una matriz cuadrada A.
    Devuelve L (con diagonal unitaria) y U.
    """
    n = A.shape[0]
    L = np.eye(n) # Matriz identidad (diagonal unitaria)
    U = A.copy()

    for k in range(n-1):
        for i in range(k+1, n):
            if U[k,k] == 0:
                raise ValueError("División por cero - matriz no factorizable")
            L[i,k] = U[i,k] / U[k,k]
            for j in range(k, n):
                U[i,j] = U[i,j] - L[i,k] * U[k,j]

    return L, U

```


$$\begin{bmatrix} 2 & -1 & 1 \\ 3 & 3 & 9 \\ 3 & 3 & 5 \end{bmatrix}$$

```
In [17]: A_a = np.array([
    [2, -1, 1],
    [3, 3, 9],
    [3, 3, 5]
])

L_a, U_a = lu_decomposition(A_a)

print("Matriz (a):")
print("L = \n", L_a)
print("U = \n", U_a)
print("Verificación (LU == A): \n", np.allclose(L_a @ U_a, A_a))
```

```
Matriz (a):
L =
[[1.  0.  0. ]
 [1.5 1.  0. ]
 [1.5 1.  1. ]]
U =
[[ 2 -1  1]
 [ 0  4  7]
 [ 0  0 -4]]
Verificación (LU == A):
False
```

$$\begin{bmatrix} 1.012 & -2.132 & 3.104 \\ -2.132 & 4.096 & -7.013 \\ 3.104 & -7.013 & 0.014 \end{bmatrix}$$

```
In [18]: A_b = np.array([
    [1.012, -2.132, 3.104],
    [-2.132, 4.096, -7.013],
    [3.104, -7.013, 0.014]
])

L_b, U_b = lu_decomposition(A_b)

print("\nMatriz (b):")
print("L = \n", np.round(L_b, 4))
print("U = \n", np.round(U_b, 4))
print("Verificación: ", np.allclose(L_b @ U_b, A_b))
```

```

Matriz (b):
L =
[[ 1.      0.      0.    ]
 [-2.1067  1.      0.    ]
 [ 3.0672  1.1978  1.    ]]
U =
[[ 1.012 -2.132  3.104 ]
 [ 0.    -0.3955 -0.4737]
 [ 0.     0.    -8.9391]]
Verificación: True

```

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 1 & 1.5 & 0 & 0 \\ 0 & -3 & 0.5 & 0 \\ 2 & -2 & 1 & 1 \end{bmatrix}$$

```

In [19]: A_c = np.array([
    [2, 0, 0, 0],
    [1, 1.5, 0, 0],
    [0, -3, 0.5, 0],
    [2, -2, 1, 1]
])

# Para matrices triangulares inferiores, L = A y U = I
L_c = A_c
U_c = np.eye(4)

print("\nMatriz (c):")
print("L = \n", L_c)
print("U = \n", U_c)
print("Verificación: ", np.allclose(L_c @ U_c, A_c))

```

```

Matriz (c):
L =
[[ 2.  0.  0.  0.]
 [ 1.  1.5 0.  0.]
 [ 0. -3.  0.5 0.]
 [ 2. -2.  1.  1.]]
U =
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
Verificación: True

```

$$\begin{bmatrix} 2.1756 & 4.0231 & -2.1732 & 5.1967 \\ -4.0231 & 6.0000 & 0 & 1.1973 \\ -1.0000 & -5.2107 & 1.1111 & 0 \\ 6.0235 & 7.0000 & 0 & -4.1561 \end{bmatrix}$$

```

In [20]: A_d = np.array([
    [2.1756, 4.0231, -2.1732, 5.1967],
    [-4.0231, 6.0000, 0, 1.1973],
    [-1.0000, -5.2107, 1.1111, 0],
    [6.0235, 7.0000, 0, -4.1561]
])

```

```

    [6.0235, 7.0000, 0, -4.1561]
])

L_d, U_d = lu_decomposition(A_d)

print("\nMatriz (d):")
print("L = \n", np.round(L_d, 4))
print("U = \n", np.round(U_d, 4))
print("Verificación: ", np.allclose(L_d @ U_d, A_d, atol=1e-8))

```

Matriz (d):

```

L =
[[ 1.      0.      0.      0.    ]
 [-1.8492  1.      0.      0.    ]
 [-0.4596 -0.2501  1.      0.    ]
 [ 2.7687 -0.3079 -5.3523  1.    ]]

```

```

U =
[[ 2.1756  4.0231 -2.1732  5.1967]
 [ 0.      13.4395 -4.0187 10.807 ]
 [ 0.      0.      -0.893  5.0917]
 [ 0.      0.      0.      12.0361]]

```

Verificación: True

Ejercicio 7

Modifique el algoritmo de eliminación gaussiana de tal forma que se pueda utilizar para resolver un sistema lineal usando la descomposición LU y, a continuación, resuelva los siguientes sistemas lineales.

```

In [21]: import numpy as np

def lu_decomposition(A):
    """
    Realiza la descomposición LU de una matriz A con pivoteo parcial.
    Devuelve P (matriz de permutación), L (triangular inferior con diagonal 1) y
    """
    n = A.shape[0]
    L = np.eye(n)
    U = A.copy()
    P = np.eye(n)

    for k in range(n-1):
        # Pivoteo parcial
        max_row = np.argmax(np.abs(U[k:, k])) + k
        if max_row != k:
            U[[k, max_row]] = U[[max_row, k]]
            P[[k, max_row]] = P[[max_row, k]]
            if k > 0:
                L[[k, max_row], :k] = L[[max_row, k], :k]

        if U[k, k] == 0:
            raise ValueError("Matriz es singular")

        for i in range(k+1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k:] -= L[i, k] * U[k, k:]

    return P, L, U

```

```
def solve_lu(P, L, U, b):
    """
    Resuelve el sistema Ax = b usando la descomposición LU
    """
    # Aplicar permutación al vector b
    Pb = P @ b

    # Resolver Ly = Pb (sustitución hacia adelante)
    n = len(Pb)
    y = np.zeros(n)
    for i in range(n):
        y[i] = Pb[i] - np.dot(L[i, :i], y[:i])

    # Resolver Ux = y (sustitución hacia atrás)
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        if U[i, i] == 0:
            raise ValueError("Matriz es singular")
        x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]

    return x
```

$$2x_1 - x_2 + x_3 = -1$$

$$3x_1 + 3x_2 + 9x_3 = 0$$

$$3x_1 + 3x_2 + 5x_3 = 4$$

```
In [24]: # Sistema (a)
A_a = np.array([
    [2, -1, 1],
    [3, 3, 9],
    [3, 3, 5]
])
b_a = np.array([-1, 0, 4])

# P_a, L_a, U_a = lu_decomposition(A_a)
# x_a = solve_lu(P_a, L_a, U_a, b_a)

print("Solución del sistema (a):")
print(f"x1 = {x_a[0]:.4f}, x2 = {x_a[1]:.4f}, x3 = {x_a[2]:.4f}")
```

Solución del sistema (a):

$x_1 = -3.0000$, $x_2 = 3.0000$, $x_3 = 1.0000$

$$1.012x_1 - 2.132x_2 + 3.104x_3 = 1.984$$

$$-2.132x_1 + 4.096x_2 - 7.013x_3 = -5.049$$

$$3.104x_1 - 7.013x_2 + 0.014x_3 = -3.895$$

```
In [25]: # Sistema (b)
A_b = np.array([
    [1.012, -2.132, 3.104],
    [-2.132, 4.096, -7.013],
    [3.104, -7.013, 0.014]
```

```

])
b_b = np.array([1.984, -5.049, 3.895]) # Ejemplo de vector b

P_b, L_b, U_b = lu_decomposition(A_b)
x_b = solve_lu(P_b, L_b, U_b, b_b)

print("\nSolución del sistema (b):")
print(f"x1 = {x_b[0]:.4f}, x2 = {x_b[1]:.4f}, x3 = {x_b[2]:.4f}")

```

Solución del sistema (b):

$x_1 = 5.8719$, $x_2 = 2.0438$, $x_3 = 0.1286$

$$\begin{aligned}
 2x_1 &= 3 \\
 x_1 + 1.5x_2 &= 4.5 \\
 -3x_2 + 0.5x_3 &= -6.6 \\
 2x_1 - 2x_2 + x_3 + x_4 &= 0.8
 \end{aligned}$$

```

In [ ]: # Sistema (c)
A_b = np.array([
    [2, 0, 0, 0],
    [1, 1.5, 0, 0],
    [0, -3, 0.5, 0],
    [2, -2, 1, 1]
])
b_b = np.array([3, 4.5, -6.6, 0.8])

P_b, L_b, U_b = lu_decomposition(A_b)
x_b = solve_lu(P_b, L_b, U_b, b_b)

print("\nSolución del sistema (b):")
print(f"x1 = {x_b[0]:.4f}, x2 = {x_b[1]:.4f}, x3 = {x_b[2]:.4f}")

```

Solución del sistema (b):

$x_1 = 1.5000$, $x_2 = 2.0000$, $x_3 = -1.2000$

$$\begin{aligned}
 2.1756x_1 + 4.0231x_2 - 2.1732x_3 + 5.1967x_4 &= 17.102 \\
 -4.0231x_1 + 6.0000x_2 + 1.1973x_4 &= -6.1593 \\
 -1.0000x_1 - 5.2107x_2 + 1.1111x_3 &= 3.0004 \\
 6.0235x_1 + 7.0000x_2 - 4.1561x_4 &= 0.0000
 \end{aligned}$$

```

In [27]: # Sistema (d)
A_b = np.array([
    [2.1756, 4.0231, -2.1732, 5.1967],
    [-4.0231, 6.0000, 0, 1.1973],
    [-1.0000, -5.2107, 1.1111, 0],
    [6.0235, 7, 0, -4.1561]
])
b_b = np.array([17.102, -6.1593, 3.0004, 0])

P_b, L_b, U_b = lu_decomposition(A_b)
x_b = solve_lu(P_b, L_b, U_b, b_b)

```

```
print("\nSolución del sistema (b):")  
print(f"x1 = {x_b[0]:.4f}, x2 = {x_b[1]:.4f}, x3 = {x_b[2]:.4f}")
```

Solución del sistema (b):

$x_1 = 2.9399$, $x_2 = 0.0707$, $x_3 = 5.6777$